

Workshop on Advanced Sampling Methodologies

Introduction to R Programming

Dr. Gianluca Boo, WorldPop, University of Southampton

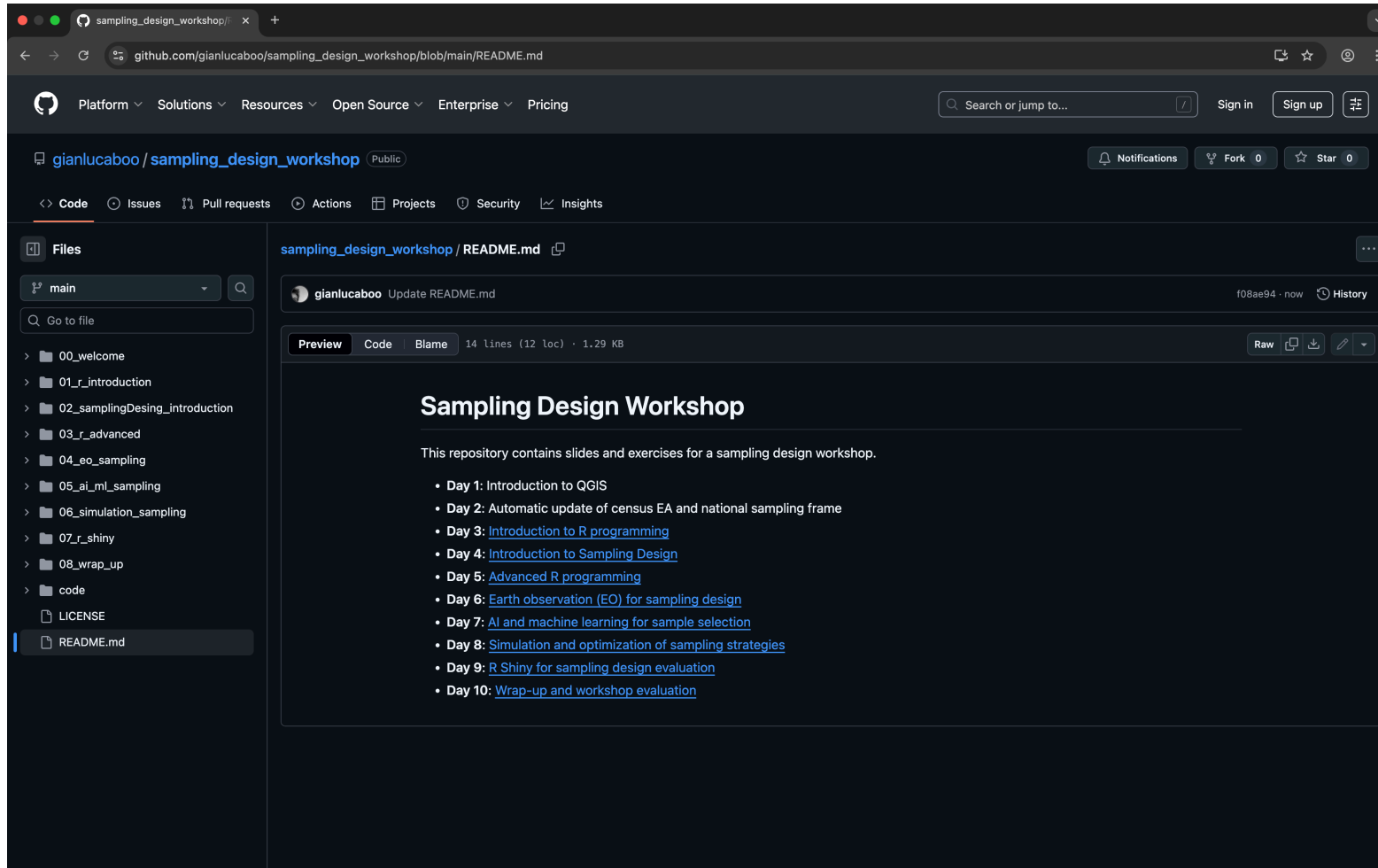
2025-09-17

Workshop's agenda

- **Day 1:** Introduction to QGIS
- **Day 2:** Automatic update of census EA and national sampling frame
- **Day 3:** Introduction to R programming
- **Day 4:** Introduction to Sampling Design
- **Day 5:** Advanced R programming
- **Day 6:** Earth observation (EO) for sampling design
- **Day 7:** AI and machine learning for sample selection
- **Day 8:** Simulation and optimization of sampling strategies
- **Day 9:** R Shiny for sampling design evaluation
- **Day 10:** Wrap-up and workshop evaluation

Course materials

All the course materials are available on a dedicated GitHub repository.



The screenshot displays the GitHub interface for the repository `gianlucaboo/sampling_design_workshop`. The left sidebar shows the file structure with folders for each day of the workshop (00_welcome to 08_wrap_up) and a `code` folder. The main content area shows the `README.md` file, which contains the following text:

Sampling Design Workshop

This repository contains slides and exercises for a sampling design workshop.

- Day 1: Introduction to QGIS
- Day 2: Automatic update of census EA and national sampling frame
- Day 3: [Introduction to R programming](#)
- Day 4: [Introduction to Sampling Design](#)
- Day 5: [Advanced R programming](#)
- Day 6: [Earth observation \(EO\) for sampling design](#)
- Day 7: [AI and machine learning for sample selection](#)
- Day 8: [Simulation and optimization of sampling strategies](#)
- Day 9: [R Shiny for sampling design evaluation](#)
- Day 10: [Wrap-up and workshop evaluation](#)

Course materials

Today's agenda

- Introduction to R and RStudio
- Data types and data structures
- Working with scripts and the console
- Importing data
- Saving and closing sessions
- Practical exercises

What is R?

Why R?

- Powerful tool for **data analysis and visualization**
- Encourages **reproducibility** and **sharing**
- Thousands of user-created **packages** extend its functionality
- Large and active **community** with extensive support

Iris k-means clustering

X Variable

Sepal.Length ▼

Y Variable

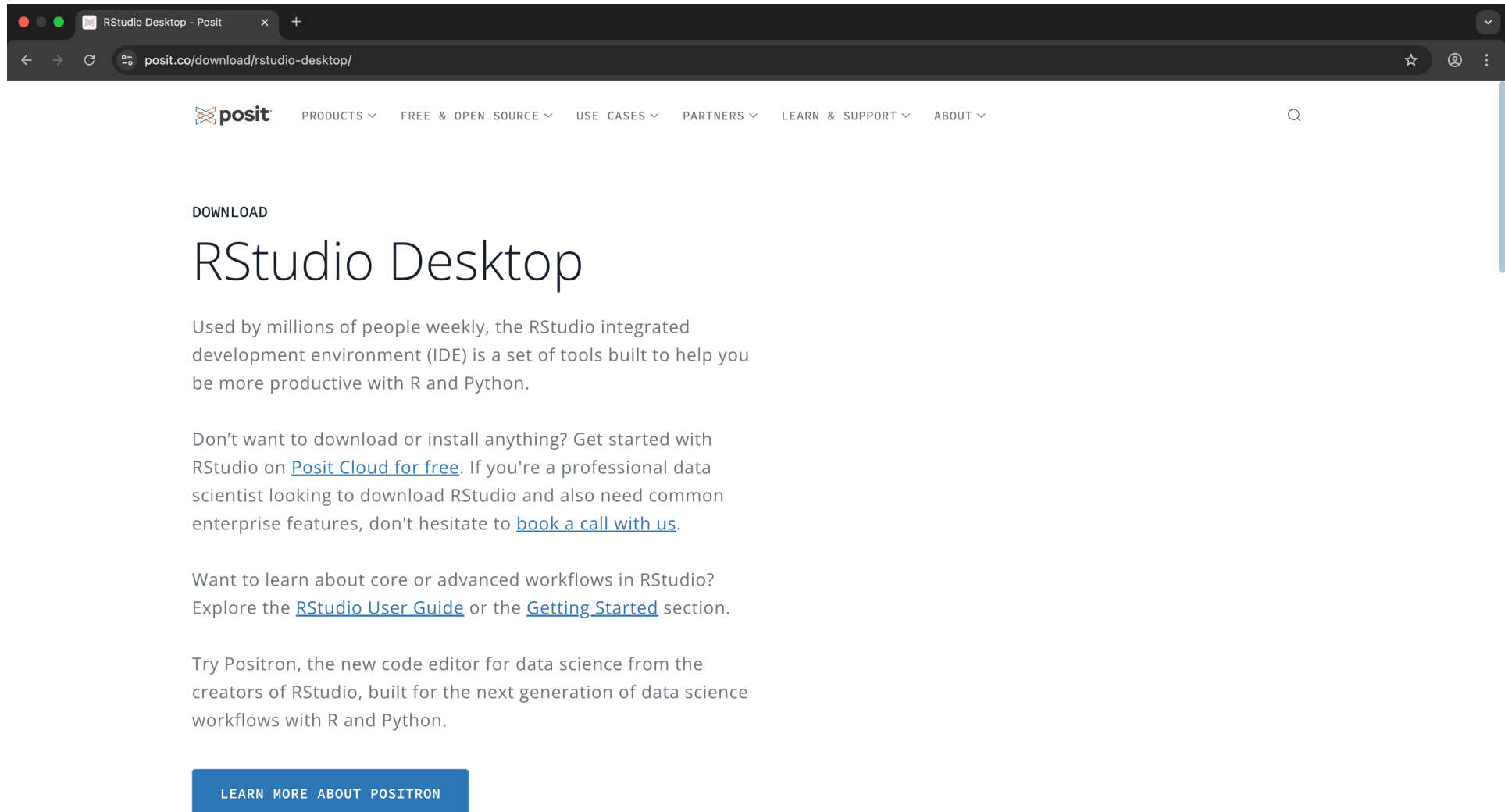
Sepal.Width ▼

Cluster count

3

Installing R and RStudio

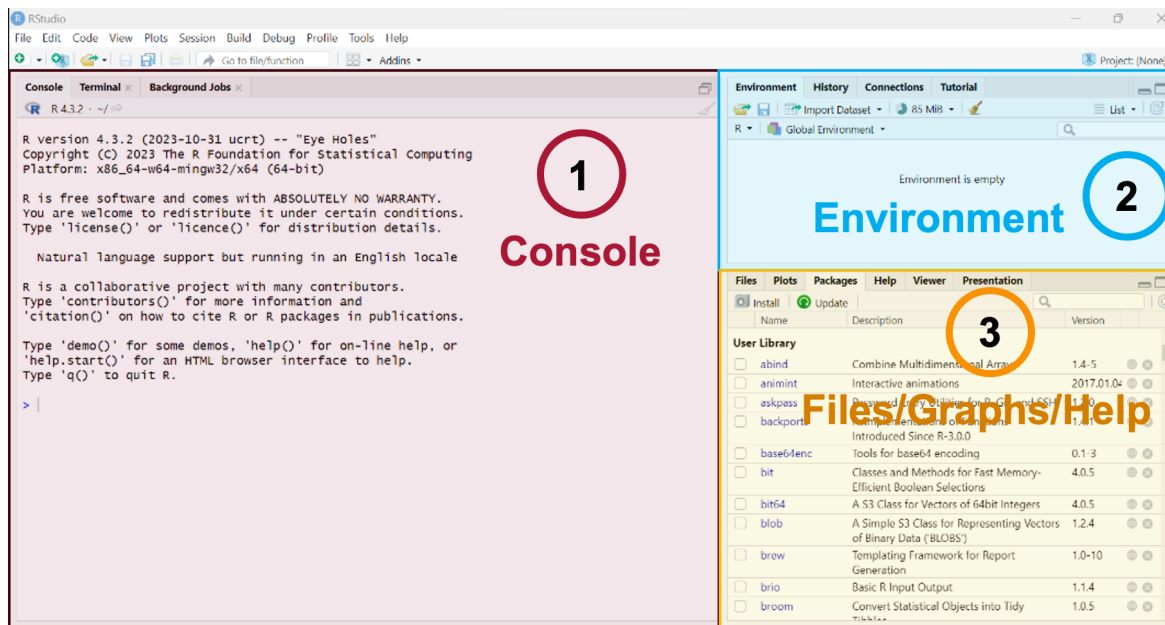
- Download and install **R and RStudio** from posit.co



RStudio interface

RStudio has four main panes:

- **Source Editor** – write and save code
- **Console** – run commands interactively
- **Environment/History** – see and manage objects
- **Files/Plots/Packages/Help/Viewer** – tools and outputs



 Try to navigate through the different panes.

Script Editor vs Console

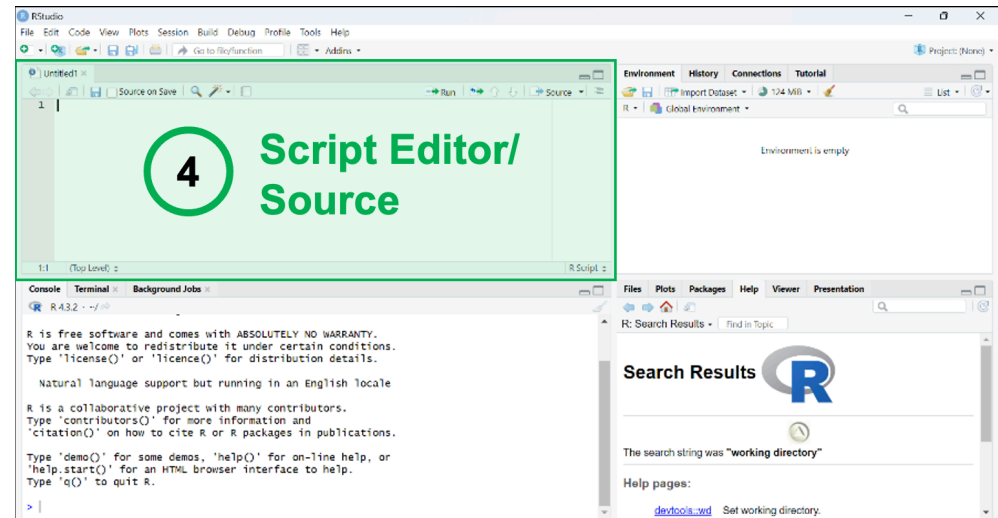
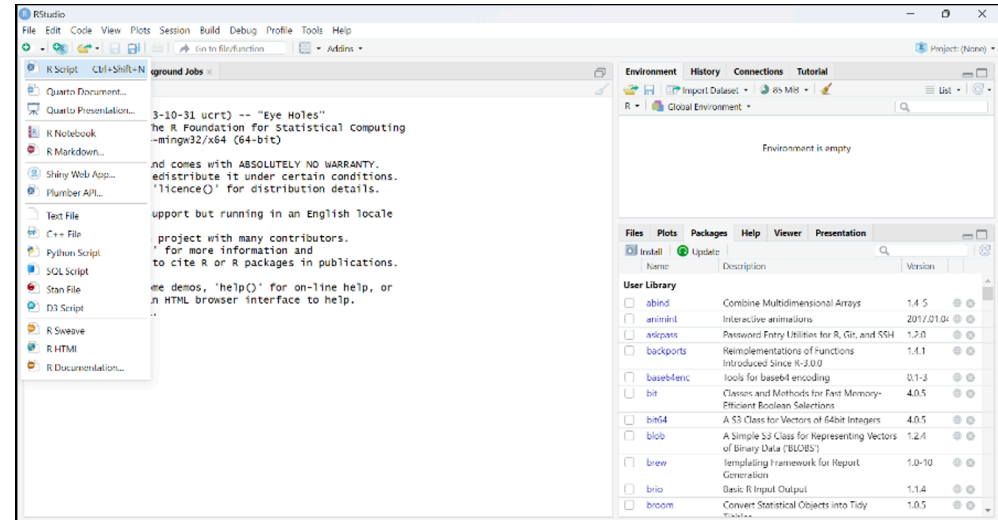
Script Editor

- Write, edit, and save code
- Organize analysis in reproducible scripts
- Prevent accidental execution before editing

Console

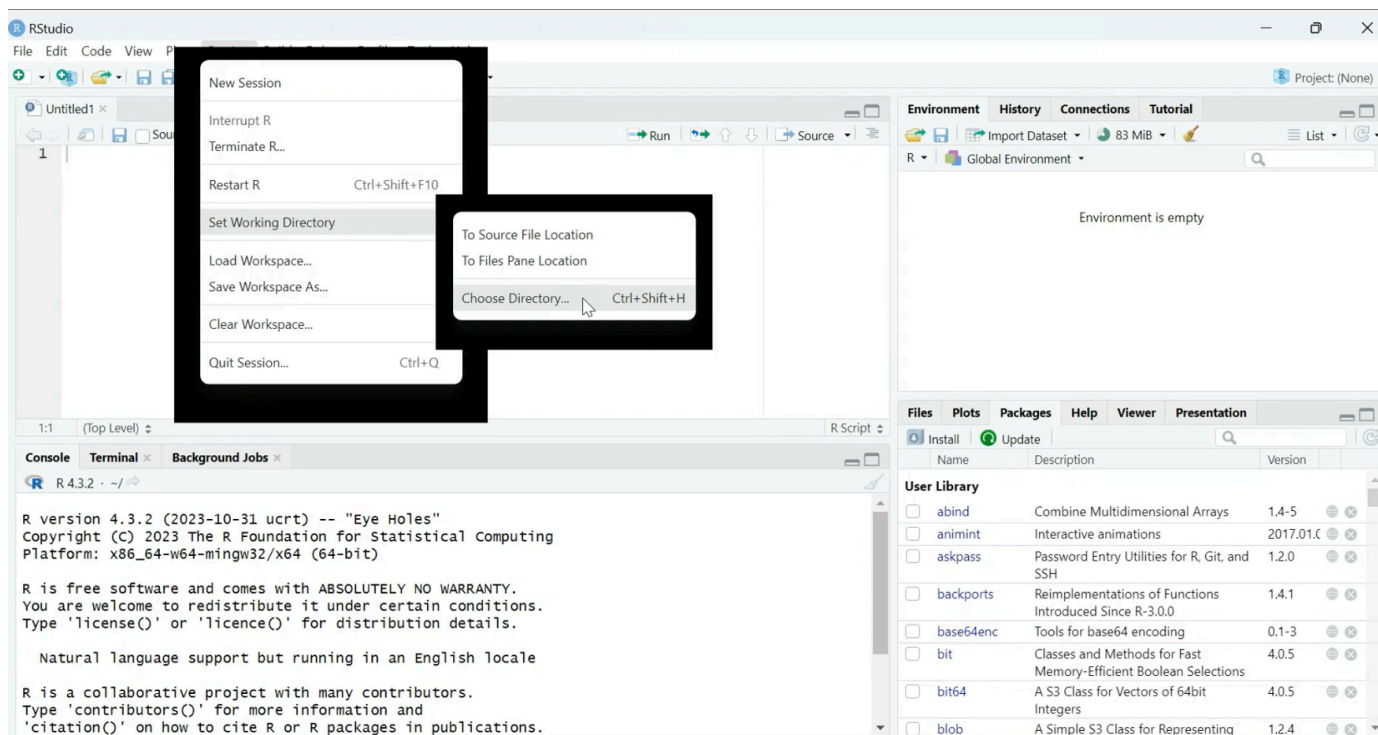
- Executes code immediately
- Use for quick checks and interactive testing
- Output and errors appear here

 Best practice: write in the Script Editor, run via Console.



Working Directory

- Your working directory is the folder where R looks for and saves files.
- Set with: `setwd("path")`
- Check with: `getwd()`
- Good practice: create a dedicated folder for each project.



 Try to run the code in your R Console and in the Script Editor.

R as a calculator

- R can perform arithmetic directly

```
1 1 + 2 * 8
2 a <- log(10)
3 b <- 5
4 b + b
```

- Results in Console are prefixed with [1] (index of first element).
- [2], [3], ... appear if output spans multiple lines.

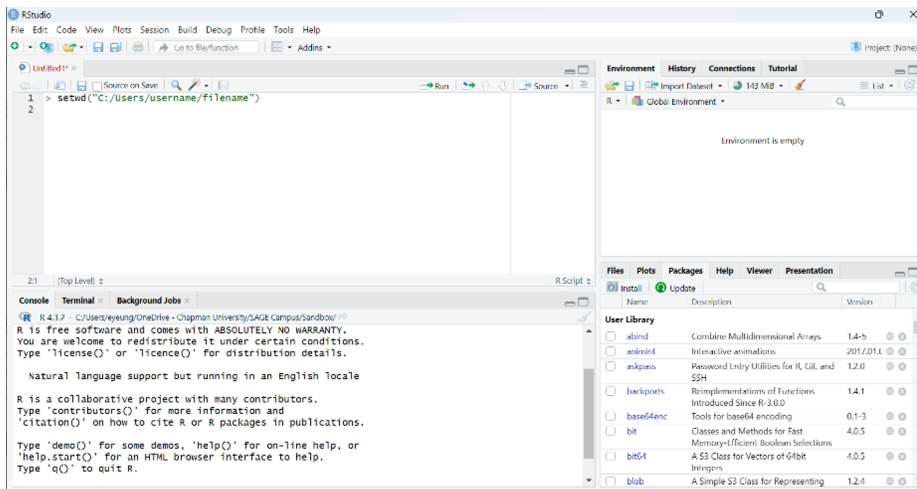
 Try to run the code in your R Console and in the Script Editor.

R packages

- Collections of functions, datasets, and documentation.
- Expand R beyond base functions.
- Install from CRAN, GitHub, or other repositories.

In RStudio

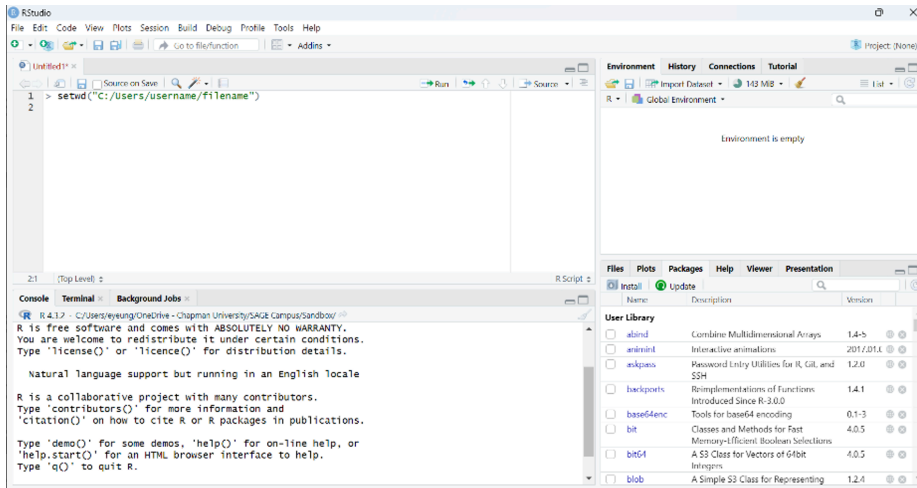
- Use the *Packages* tab to install, update, or remove.
- Or install in the Script Editor



1 `install.packages("dplyr")`

Loading packages

- Load packages into a session via checkbox in the *Packages* pane.



- Or load packages into a session with:

```
1 library(dplyr)
```

 Best practice: only load what you need.

R objects

- Everything in R is an **object** (vectors, lists, functions, data frames, etc.).
- Assign values using `<-` (**Alt** + `-` (minus sign) on Windows and Linux, and **Option** + `-` on a Mac).

```
1 x <- 5
2 x + x    # returns 10
```

Naming tips

-  Descriptive and concise
-  Use underscores `_` or periods `.`
-  Avoid very long names
-  Avoid existing function names
-  No blank spaces

 Try to run the code in your Script Editor.

Data types

- **Numeric:** numbers, may include decimals.

```
1 w <- c(1.7, 3)
2 x <- c(1, 2, 3) # integers are numeric without decimals
```

- **Character:** text strings.

```
1 y <- c("male", "female")
```

- **Logical:** TRUE/FALSE values.

```
1 y <- c(TRUE, FALSE)
```

- **Factor:** categorical variables.

```
1 gender <- factor(c("boy", "girl", "girl", "boy"))
```

Check type:

```
1 class(object)
2 typeof(object)
```

Data structures

- **Vectors:** one-dimensional, homogeneous elements
- **Matrices:** two-dimensional, homogeneous elements
- **Lists:** flexible, heterogeneous elements
- **Arrays:** multi-dimensional, homogeneous elements
- **Data frames:** tabular, columns may differ in type

Vectors

- The most basic structure in R
- One-dimensional, elements must be the **same type**

```
1 numbers <- c(1, 2, 3, 4, 5)
2 colors <- c("red", "green", "blue")
3 logical_values <- c(TRUE, FALSE, TRUE)
```

Indexing with brackets:

```
1 numbers[1]      # first element
2 colors[2:3]     # second and third elements
```

Vectorized operations apply to all elements:

```
1 numbers * 2     # multiplies each element by 2
```

 Try to run the code in your Script Editor.

Matrices

- Two-dimensional, **homogeneous elements**
- Create with `matrix()`

```
1 rownames <- c("math", "science", "history", "English", "law")
2 colnames <- c("male", "female")
3 subject_matrix <- matrix(
4   c(10,9,11,13,12,18,17,13,6,5),
5   nrow = 5, ncol = 2, byrow = TRUE,
6   dimnames = list(rownames, colnames)
7 )
```

- Add row: `rbind()`
- Add column: `cbind()`

 Try to add a row to `subject_matrix` for `geography <- c(4,2)` using `rbind` in your Script Editor.

Matrices to Data Frames

- Convert a matrix into a data frame to **mix numeric and non-numeric types**.

```
1 matrix_stats <- matrix(  
2   c(4015, 3980, 3756, 3333, 4000, 3000, 2000, 1000),  
3   nrow = 4, ncol = 2, byrow = TRUE,  
4   dimnames = list(  
5     c("freshmen", "sophomores", "juniors", "seniors"),  
6     c("enrollment", "living on campus")  
7   )  
8 )  
9  
10 df_stats <- as.data.frame(matrix_stats)  
11 avg.grades <- c("A-", "B+", "B-", "A+")  
12 school_stats <- cbind(df_stats, avg.grades)
```

 Try to run the code in your Script Editor.

Data Frames

- Tabular structure with columns of different types
- Similar to spreadsheets

```
1 cats <- data.frame(  
2   coat = c("Persian", "black", "tabby"),  
3   weight = c(2.1, 5.0, 3.2),  
4   likes_string = c(TRUE, FALSE, TRUE)  
5 )  
6  
7 animal_sleep <- data.frame(  
8   ranking = c(4, 1, 2, 3),  
9   animal = c("koala", "capybara", "camel", "panda"),  
10  country = c("Australia", "Brazil", "Egypt", "China"),  
11  avg_sleep_hr = c(20, 3, 6, 12)  
12 )
```

 Try to run the code in your Script Editor.

Lists

- Flexible data structure with elements of **different types**
- Can include vectors, data frames, matrices, or other lists

```
1 my_list <- list(  
2   name = "Alice",  
3   age = 25,  
4   scores = c(90, 85, 92),  
5   passed = TRUE  
6 )  
7  
8 my_list$name  
9 my_list[[2]]
```

 Try to run the code in your Script Editor.

Arrays

- Extend vectors to two or more dimensions
- All elements must be the same type

```
1 arr <- array(1:12, dim = c(2, 3, 2))
2 arr
3 arr[1, 2, 1] # element at row 1, col 2, slice 1
```

Useful for structured multi-dimensional data. However, this is hardly used in most cases.

 Try to run the code in your Script Editor.

Importing data

- Create data manually, but usually import from files

Common formats

- **Text:** `.txt` (`readLines()` function)
- **Tabular data:** `.csv`, `.tsv` (`read.table()` function or `read_csv()` function from the `readr` package)
- **Excel:** `.xlsx` (`xlsx` package)
- **Google sheets:** (`googlesheets` package)
- **Statistics program:** SPSS, SAS (`haven` package)
- **Databases:** MySQL (`RMySQL` package)

Example:

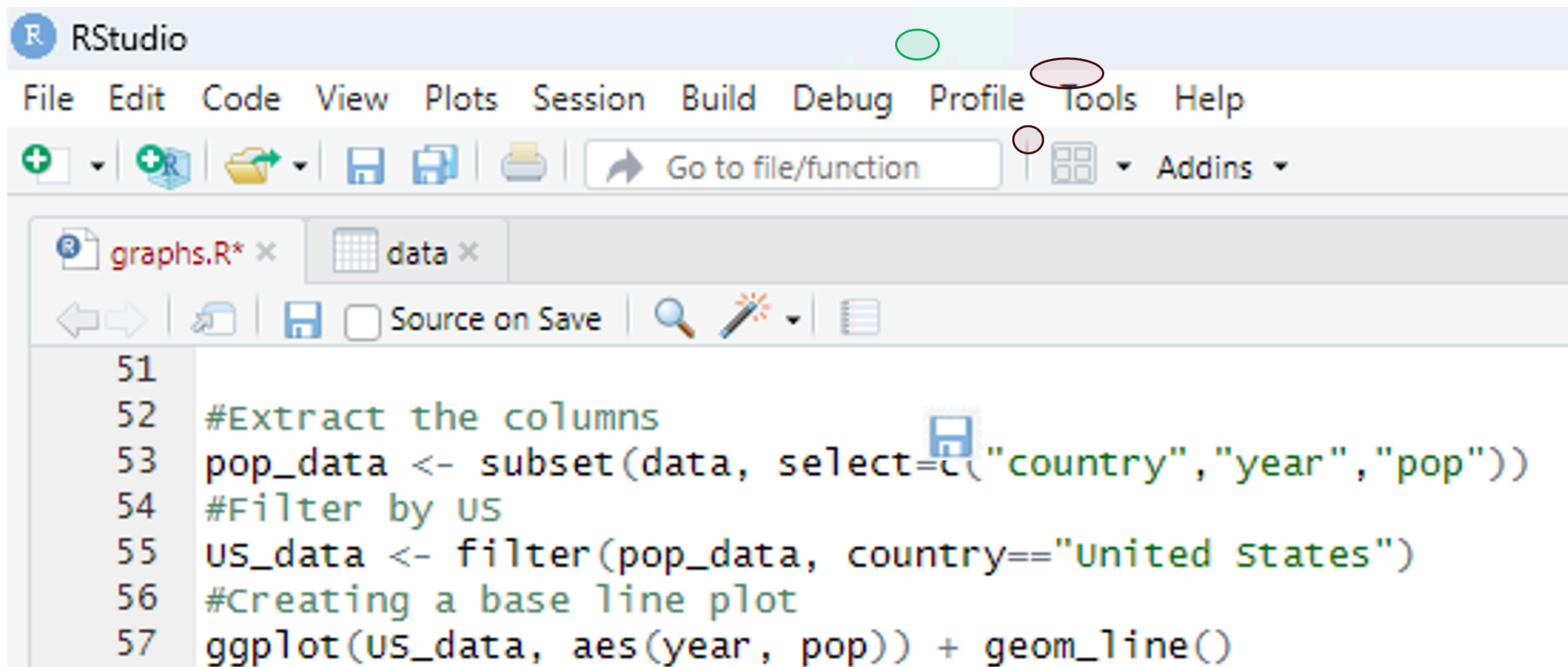
```
1 csv_data <- read_csv("data/world_data.csv")
```

➡ Always use forward slashes `/` in file paths.

i Try to run the code in your Script Editor.

Saving your script

- Save all steps of analysis in **.R** scripts
- Add comments with **#**
- Save with **File → Save** or **Ctrl+S / Cmd+S**
- Unsaved scripts show in red with an asterisk *****



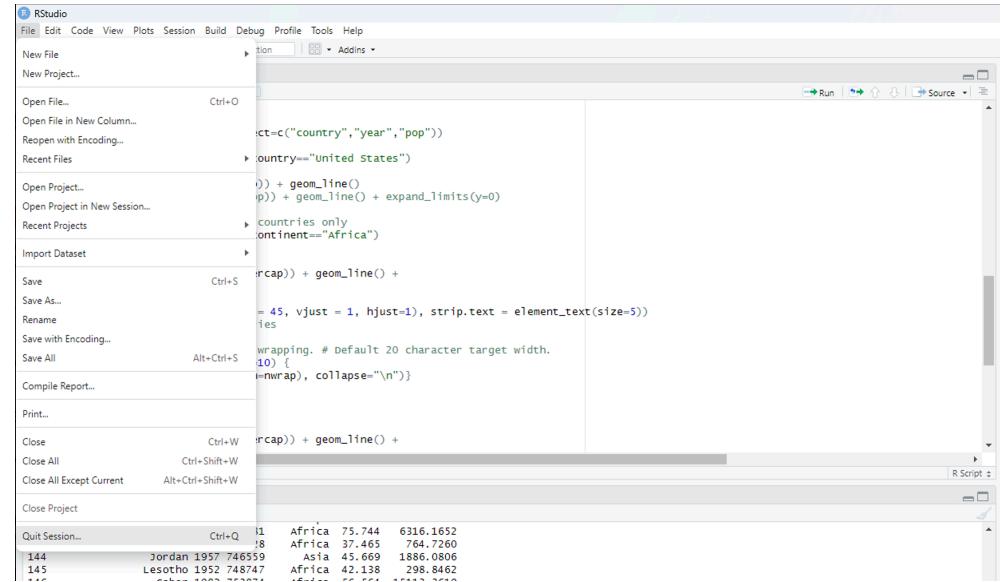
The screenshot shows the RStudio application window. The title bar says 'RStudio'. The menu bar includes 'File', 'Edit', 'Code', 'View', 'Plots', 'Session', 'Build', 'Debug', 'Profile', 'Tools', and 'Help'. The toolbar contains icons for creating a new file, opening a file, saving, and a search bar labeled 'Go to file/function'. Below the toolbar, there are two tabs: 'graphs.R*' and 'data'. The 'graphs.R*' tab is active and its title is red, indicating it is unsaved. Below the tabs is a toolbar with navigation arrows, a 'Source on Save' checkbox, and other icons. The main editor area shows R code with line numbers 51 through 57. A save icon (floppy disk) is hovering over the code on line 53.

```
51  
52 #Extract the columns  
53 pop_data <- subset(data, select=c("country", "year", "pop"))  
54 #Filter by US  
55 US_data <- filter(pop_data, country=="United states")  
56 #Creating a base line plot  
57 ggplot(US_data, aes(year, pop)) + geom_line()
```

Closing an R Session

Options:

- **File** → **Quit Session**
- `q()` in Console



➡ Do **not** save the workspace image. Always start fresh for reproducibility.

i Try to close and reopen your R Session.

R cheatsheets

- R and RStudio are relatively difficult to learn at the beginning
- Once you master the basis it is much easier to move forward
- R cheatsheets are an excellent resource for beginners

Base R Cheat Sheet

Getting Help

Accessing the help files

?mean
Get help of a particular function.
help.search('weighted mean')
Search the help files for a word or phrase.
help(package = 'dplyr')
Find help for a package.

More about an object

str(iris)
Get a summary of an object's structure.
class(iris)
Find the class an object belongs to.

Using Packages

install.packages('dplyr')
Download and install a package from CRAN.

library(dplyr)
Load the package into the session, making all its functions available to use.

dplyr::select
Use a particular function from a package.

data(iris)
Load a built-in dataset into the environment.

Working Directory

getwd()
Find the current working directory (where inputs are found and outputs are sent).

setwd('C://file/path')
Change the current working directory.

Use projects in RStudio to set the working directory to the folder you are working in.

Vectors

Creating Vectors

<code>c(2, 4, 6)</code>	<code>2 4 6</code>	Join elements into a vector
<code>2:6</code>	<code>2 3 4 5 6</code>	An integer sequence
<code>seq(2, 3, by=0.5)</code>	<code>2.0 2.5 3.0</code>	A complex sequence
<code>rep(1:2, times=3)</code>	<code>1 2 1 2 1 2</code>	Repeat a vector
<code>rep(1:2, each=3)</code>	<code>1 1 1 2 2 2</code>	Repeat elements of a vector

Vector Functions

sort(x) Return x sorted.	rev(x) Return x reversed.
table(x) See counts of values.	unique(x) See unique values.

Selecting Vector Elements

By Position	
<code>x[4]</code>	The fourth element.
<code>x[-4]</code>	All but the fourth.
<code>x[2:4]</code>	Elements two to four.
<code>x[-(2:4)]</code>	All elements except two to four.
<code>x[c(1, 5)]</code>	Elements one and five.
By Value	
<code>x[x == 10]</code>	Elements which are equal to 10.
<code>x[x < 0]</code>	All elements less than zero.
<code>x[x %in% c(1, 2, 5)]</code>	Elements in the set 1, 2, 5.
Named Vectors	
<code>x['apple']</code>	Element with name 'apple'.

Programming

For Loop

```
for (variable in sequence){  
  Do something  
}
```

Example

```
for (i in 1:4){  
  j <- i + 10  
  print(j)  
}
```

While Loop

```
while (condition){  
  Do something  
}
```

Example

```
while (i < 5){  
  print(i)  
  i <- i + 1  
}
```

If Statements

```
if (condition){  
  Do something  
} else {  
  Do something different  
}
```

Example

```
if (i > 3){  
  print('Yes')  
} else {  
  print('No')  
}
```

Functions

```
function_name <- function(var){  
  Do something  
  return(new_variable)  
}
```

Example

```
square <- function(x){  
  squared <- x*x  
  return(squared)  
}
```

Reading and Writing Data

Also see the **readr** package.

Input	Output	Description
<code>df <- read.table('file.txt')</code>	<code>write.table(df, 'file.txt')</code>	Read and write a delimited text file.
<code>df <- read.csv('file.csv')</code>	<code>write.csv(df, 'file.csv')</code>	Read and write a comma separated value file. This is a special case of read table/ write table.
<code>load('file.Rdata')</code>	<code>save(df, file = 'file.Rdata')</code>	Read and write an R data file, a file type special for R.

Conditions	a == b	Are equal	a > b	Greater than	a >= b	Greater than or equal to	is.na(a)	Is missing
	a != b	Not equal	a < b	Less than	a <= b	Less than or equal to	is.null(a)	Is null

Take home

- R is a **versatile tool for data analysis and visualization** – free, open-source, and supported by a vast community with thousands of packages.
- RStudio **enhances productivity** – organize, edit, and execute code efficiently using the Script Editor, Console, and built-in panes.
- Understand **R objects, data types, and structures** – Vectors, matrices, lists, data frames, and arrays form the building blocks of R programming.
- **Reproducibility** matters – save scripts, comment your code, and avoid saving workspace images to ensure analyses can be repeated.
- Packages **expand functionality** – install, load, and use packages to extend R's capabilities for importing, manipulating, and visualizing data.

Resources

R Core Team. (2025). R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org>

The official reference for R itself, including installation, base functions, and documentation.

RStudio Team. (2025). RStudio: Integrated Development Environment for R. <https://posit.co>

Wickham, H., & Grolemund, G. (2017). R for Data Science: Import, Tidy, Transform, Visualize, and Model Data. O'Reilly Media. <https://r4ds.had.co.nz>

Venables, W. N., Smith, D. M., & the R Core Team. (2023). An Introduction to R. CRAN. <https://cran.r-project.org/doc/manuals/r-release/R-intro.pdf>

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). An Introduction to Statistical Learning with Applications in R (2nd edition). Springer.

Exercise

Please download the R script with exercises from **GitHub** and try to complete it.